

计算理论与算法分析设计

教材:

[王] 王晓东, 计算机算法设计与分析(第4版), 电子工业.

参考资料:

[C] 潘金贵等译, Cormen等著, 算法导论, 机械工业.

[M] 黄林鹏等译, Manber著, 算法引论-一种创造性方法, 电子.

[刘] 刘汝佳等, 算法艺术与信息学竞赛, 清华大学.

第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

搜索算法

- 穷举搜索(brute-force Search)
- 图遍历(Graph traversal)
 - 深度优先搜索(DFS)
 - 广度优先搜索(BFS)
- 树遍历(Tree traversal)
 - 回溯(Backtracking)
 - 分枝限界法(Branch and Bound);
 - 博弈树搜索($\alpha - \beta$ Search)等
- 启发式搜索(Heuristic Search)

方法概述: 搜索算法介绍

- 搜索的重要性和提高效率的途径
 - 搜索是人工智能的基本手段. 推理可以看成一种搜索, 联想、回忆、灵感也是一种搜索过程;
 - 搜索效率: ***Time Complexity, Space Complexity, Solution Quality; Completeness***
 - 提高搜索的基本途径: 一是积累并恰当使用有助于减少搜索的信息和知识(启发式方法); 二是采用并行处理技术。

方法概述: 搜索算法介绍 (续)

- 搜索算法
 - (1) 穷举搜索(*Exhaustive Search*)
 - (2) 盲目搜索(*Blind Search*)
 - 深度优先(*DFS*)或回溯搜索(*Backtracking*);
 - 广度优先搜索(*BFS*);
 - 分枝限界法(*Branch & Bound*);
 - 博弈树搜索(α - β Search)
 - (3) 启发式搜索(*Heuristic Search*)

方法概述: 搜索算法介绍 (续)

- 搜索空间的三种表示
 - 表序表示: 搜索对象用线性表数据结构表示;
 - 显式图表示: 搜索对象在搜索前就用图(树)的数据结构表示;
 - 隐式图表示: 除了初始结点, 其他结点在搜索过程中动态生成. 缘于搜索空间大, 难以全部存储.

方法概述: 搜索算法介绍 (续)

- 提高搜索效率的思考: 随机搜索
 - 上世纪70年代中期开始, 国外一些学者致力于研究随机搜索求解困难的组合问题, 将随机过程引入搜索:
 - 选择规则是随机地从可选结点中取一个, 从而可以从统计角度分析搜索的平均性能:
 - 随机搜索的一个成功例子: 判定一个很大的数是不是素数, 获得了第一个多项式时间的算法:

组合爆炸

一个问题有 n 个变量, k 个选择,
则遍历空间的大小为 k^n

一张充分大的普通纸对折40次:

高度超过5千座珠穆朗玛峰

一张充分大的普通纸对折50次:

月地距离的128倍

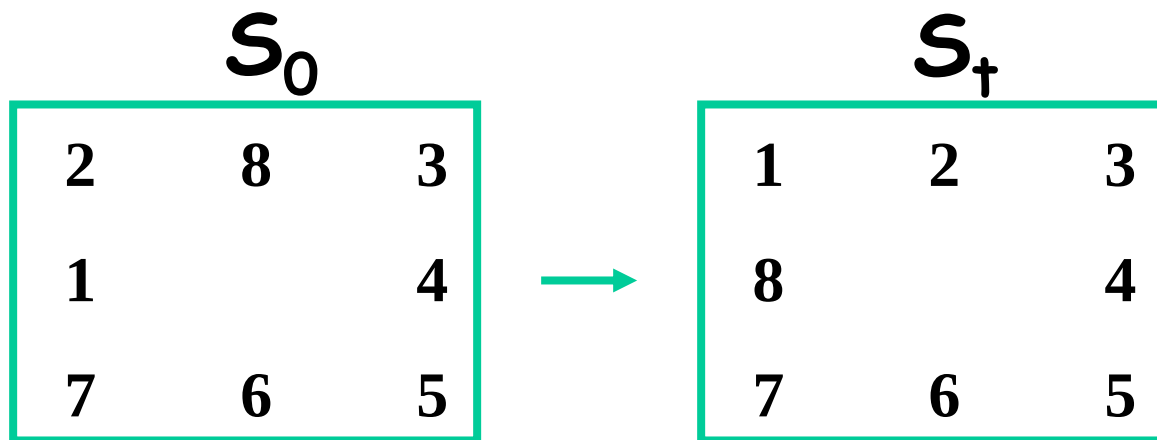
并行处理也不可能克服和绕过组合爆炸;
但并行处理可能是当前提高求解规模的
最好和现实的途径。

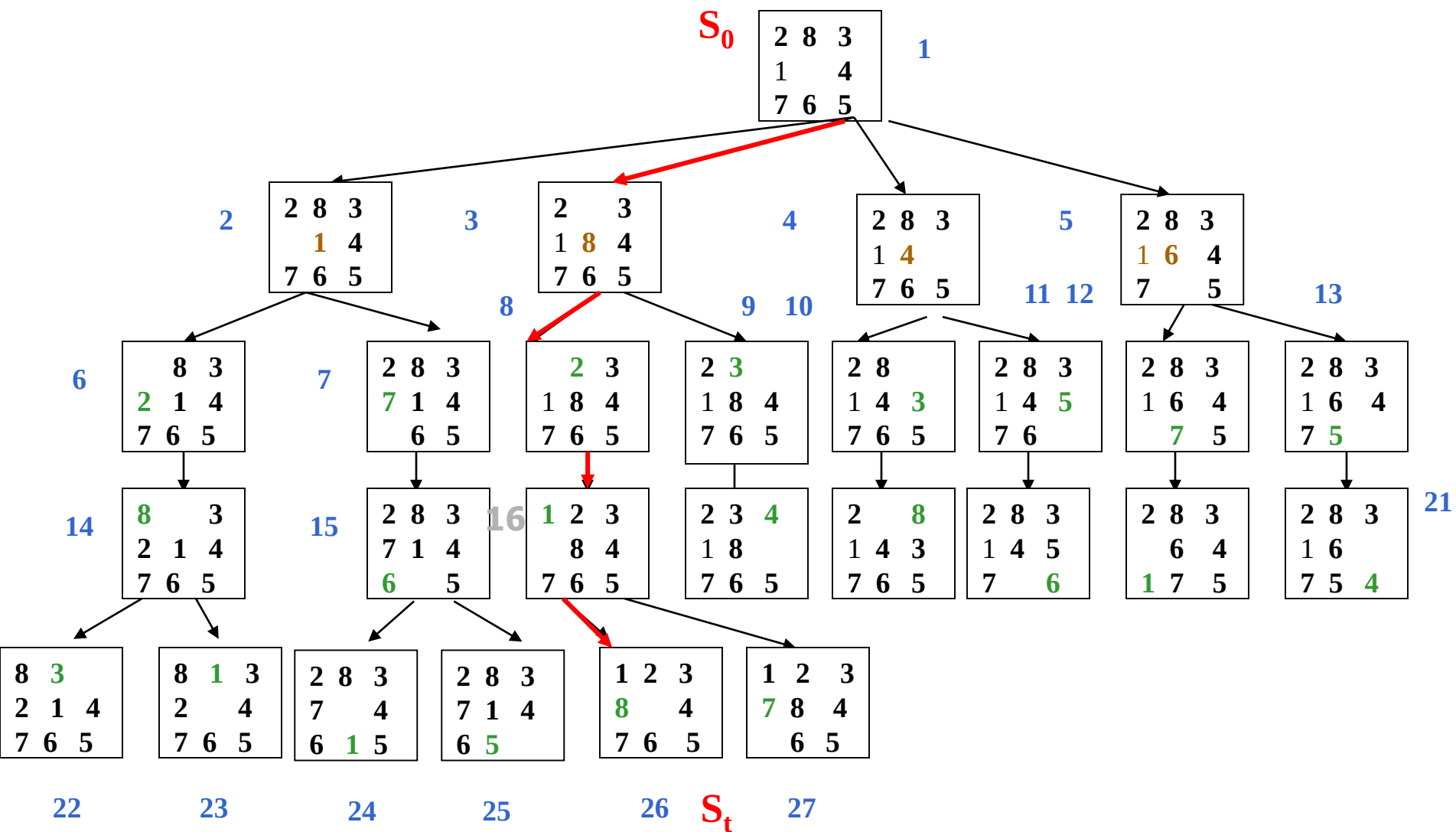
一、宽度优先搜索法

Breadth First Search (BFS)

宽度优先搜索法：逐层地对结点进行扩展。

·例：重排九宫问题：在3X3的方格棋盘上放置分别标有数字1，2，3， $\dots$ ，8的八张牌，初始状态为 S_0 ，目标状态为 S_+





Route: $S_0 \rightarrow 3 \rightarrow 8 \rightarrow 16 \rightarrow 26(S_t)$

宽度优先搜索法

- 优点：

只要问题有解，用宽度优先搜索法一定可以得到解，而且得到的是路径最短的解。

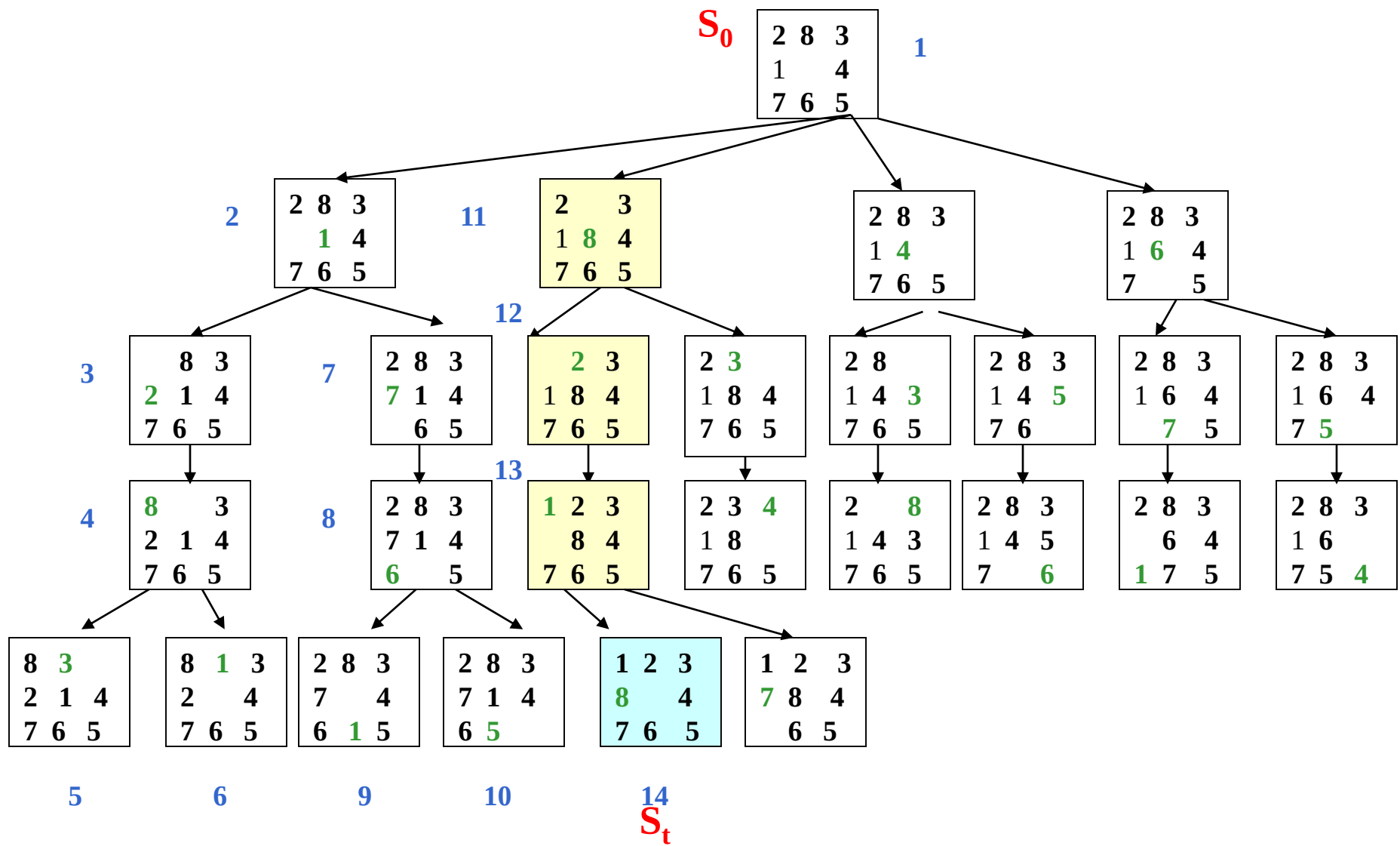
缺点：

盲目性较大。当目标结点距离初始结点较远时将会产生许多无用结点，搜索效率低。

二、深度优先搜索法

Depth First Search (DFS)

深度优先搜索法：纵深地对结点进行扩展。



$S_0: 1 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 14: S_t$

深度优先搜索法

- 搜索一旦进入某个分支，就将沿着该分支一直向下搜索。如果目标结点恰好在此分支上，则可较快地得到解。
- 但如果目标结点不在此分支上，而该分支又是一个无穷分支，则就不可能得到解。

所以深度优先搜索是不完备的。

三、有界深度优先搜索法

解决深度优先搜索不完备的问题，引入**搜索深度的界限**（设为 d_m ），即：当搜索深度达到了深度界限，而尚未出现目标结点，就换一个分支进行搜索。

四、启发式搜索法

- 问题：对于重排九宫问题如何构造估价函数？

四、启发式搜索法

估价函数：用于估价结点重要性的函数。

·一般形式： $f(x)=g(x)+h(x)$

其中： $g(x)$ 为从初始结点 S_i 到结点 x 已经付出的代价；

· $h(x)$ 是从结点 x 到目标结点 S_t 的最优路径的估计代价，它体现了问题的启发性信息。

$g(x)$ ：表示结点 x 的深度；

$h(x)$ ：表示结点 x 的格局与目标结点格局不相同的牌数。

回溯法的引入

- 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。

回溯法的引入

- 回溯法的基本做法是搜索，或是一种组织得井井有条的、能避免不必要搜索的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。
- 回溯法在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解：如果肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。

回溯法的算法框架

- 一、问题的解空间
- 二、回溯法的基本思想
- 三、递归回溯
- 四、迭代回溯
- 五、子集树与排列树

问题的解空间

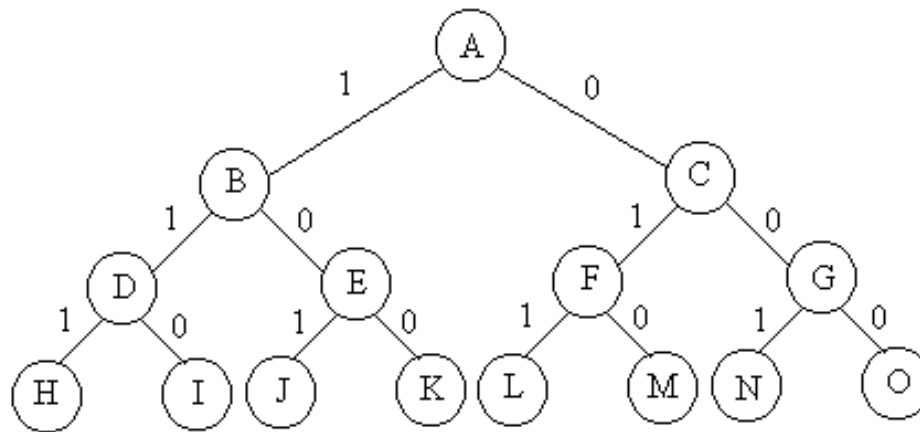
- 应用回溯法解问题时，首先应明确定义问题的解空间。问题的解空间应至少包含问题的一个(最优)解。
 - 问题的解向量：回溯法希望一个问题的解能够表示成一个 n 元式 $(x_1, x_2, \dots, x_n)$ 的形式。
 - 显约束：对分量 x_i 的取值限定。
 - 隐约束：为满足问题的解而对不同分量之间施加的约束。
 - 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。
 - 注意：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。

搜索空间的三种表示

- 穷举搜索(brute-force Search)
- 表序表示：搜索对象用线性表数据结构表示；
- 显式图表示：搜索对象在搜索前就用图(树)的数据结构表示；
- 隐式图表示：除了初始结点，其他结点在搜索过程中动态生成。缘于搜索空间大，难以全部存储。
- 灌水问题：7升和3升，量出5升。

0-1背包问题的解空间

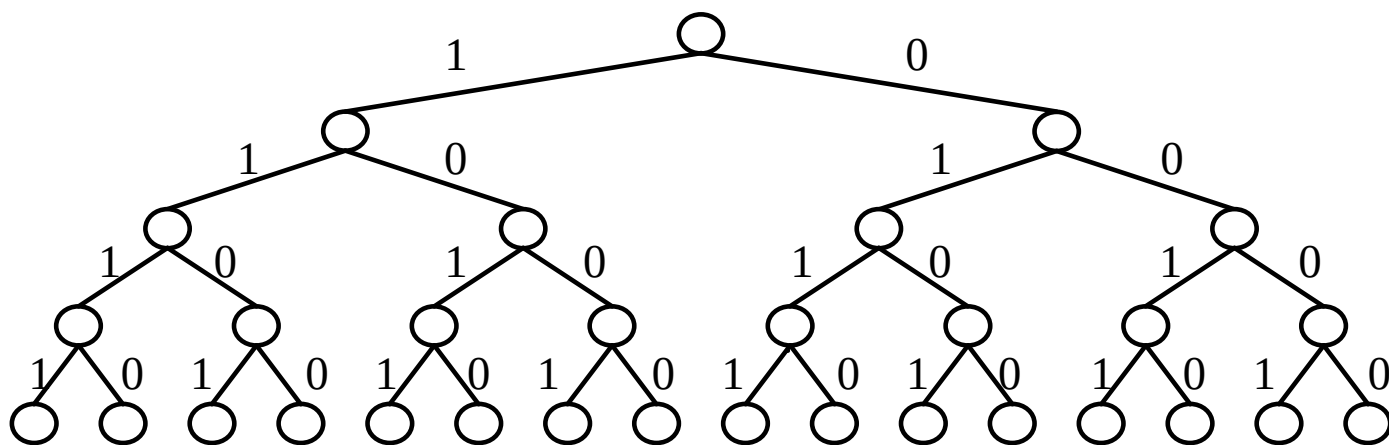
- 例如，对于有 n 种可选物品的0-1背包问题，其解空间由长度为 n 的0-1向量组成。



$n=3$ 时的0-1背包问题用完全二叉树表示的解空间

状态空间树

- 状态空间树：问题解空间的树形式表



- **n=4**时背包问题的状态空间树

空间树的动态搜索

有些问题，只要可行解，
不需要最优解，例如八后
问题和图的着色问题

- 可行解和最优解

- ✓ **可行解**：满足约束条件的解，解空间中的一个子集
- ✓ **最优解**：使目标函数取极值（极大或极小）的可行解，一个或少数几个

例：货郎担问题，有 n^n 种可能解。 $n!$ 种可行解，
只有一个或几个解是最优解。

例：背包问题， 2^n 有种可能解，有些是可行解，
只有一个或几个是最优解。

回溯法的基本思想

- 回溯法的基本步骤:

- (1) 针对所给问题, 定义问题的解空间;
- (2) 确定易于搜索的解空间结构;
- (3) 以深度优先方式搜索解空间, 并在搜索过程中用剪枝函数避免无效搜索。

- 常用剪枝函数:

- 用约束函数在扩展结点处剪去不满足约束的子树;
- 用限界函数剪去得不到最优解的子树。

生成问题状态的基本方法

- **扩展结点**：一个正在产生儿子的结点称为扩展结点
- **活结点**：一个自身已生成但其儿子还没有全部生成的节点称做活结点
- **死结点**：一个所有儿子已经产生的结点称做死结点
- **深度优先的问题状态生成法**：如果对一个扩展结点R，一旦产生了它的一个儿子C，就把C当做新的扩展结点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展结点，继续生成R的下一个儿子（如果存在）
- **宽度优先的问题状态生成法**：在一个扩展结点变成死结点之前，它一直是扩展结点

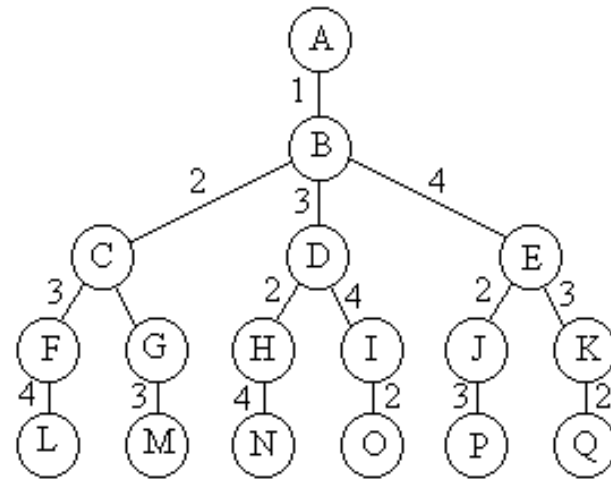
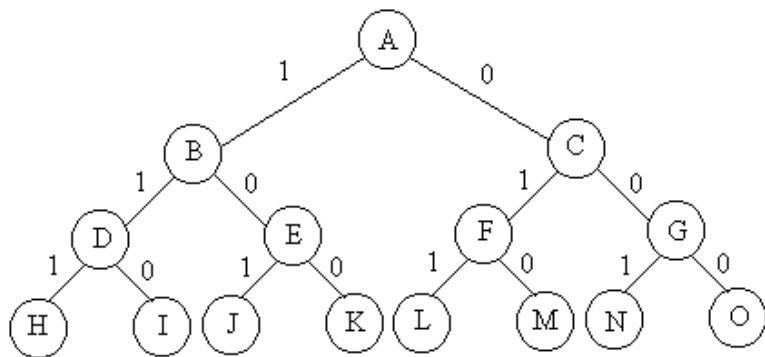
生成问题状态的基本方法

- **回溯法**：为了避免生成那些不可能产生最佳解的问题状态，要不断地利用**约束函数**来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量。
- **具有限界函数的深度优先生成法称为回溯法。**

回溯法特点

- 关于复杂性：
 - 用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。
 - 如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。

子集树与排列树



请问：每个树的叶节点数目是多少？

- 0-1背包问题
- 含有 2^n 个叶节点

- 旅行商问题
- 含有 $(n-1)!$ 个叶节点

子集树与排列树

- 子集树：当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，相应的解空间树称为子集树。
- 排列树：当所给的问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。

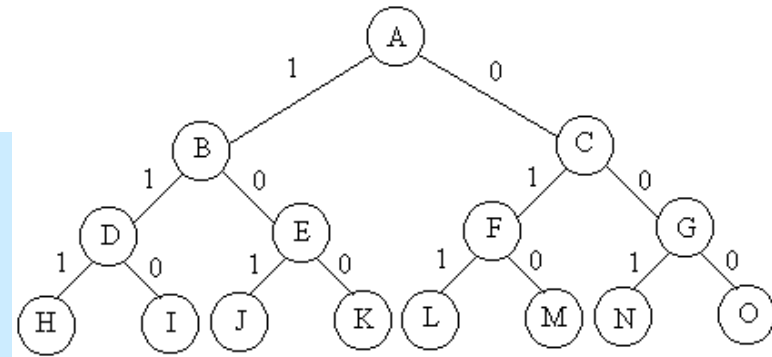
0-1背包问题的解空间是什么树？

旅行商问题的解空间是什么树？

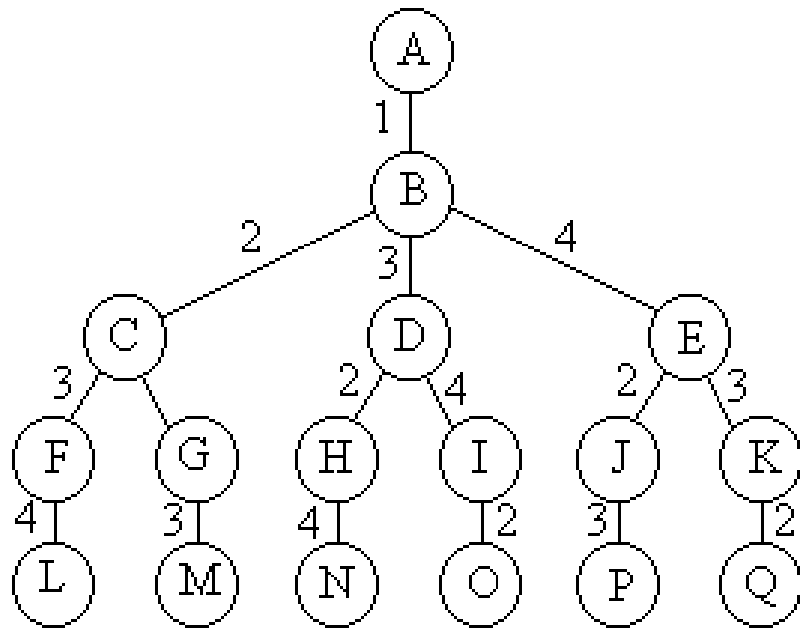
5. 子集树与排列树

- 遍历子集树需 $O(2^n)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=0;i<=1;i++) {
            x[t]=i;
            if (constraint(t)&&bound(t)) backtrack(t+1);
        }
}
```



子集树与排列树



**遍历排列树需要
 $O((n-1)!)$ 计算时间**

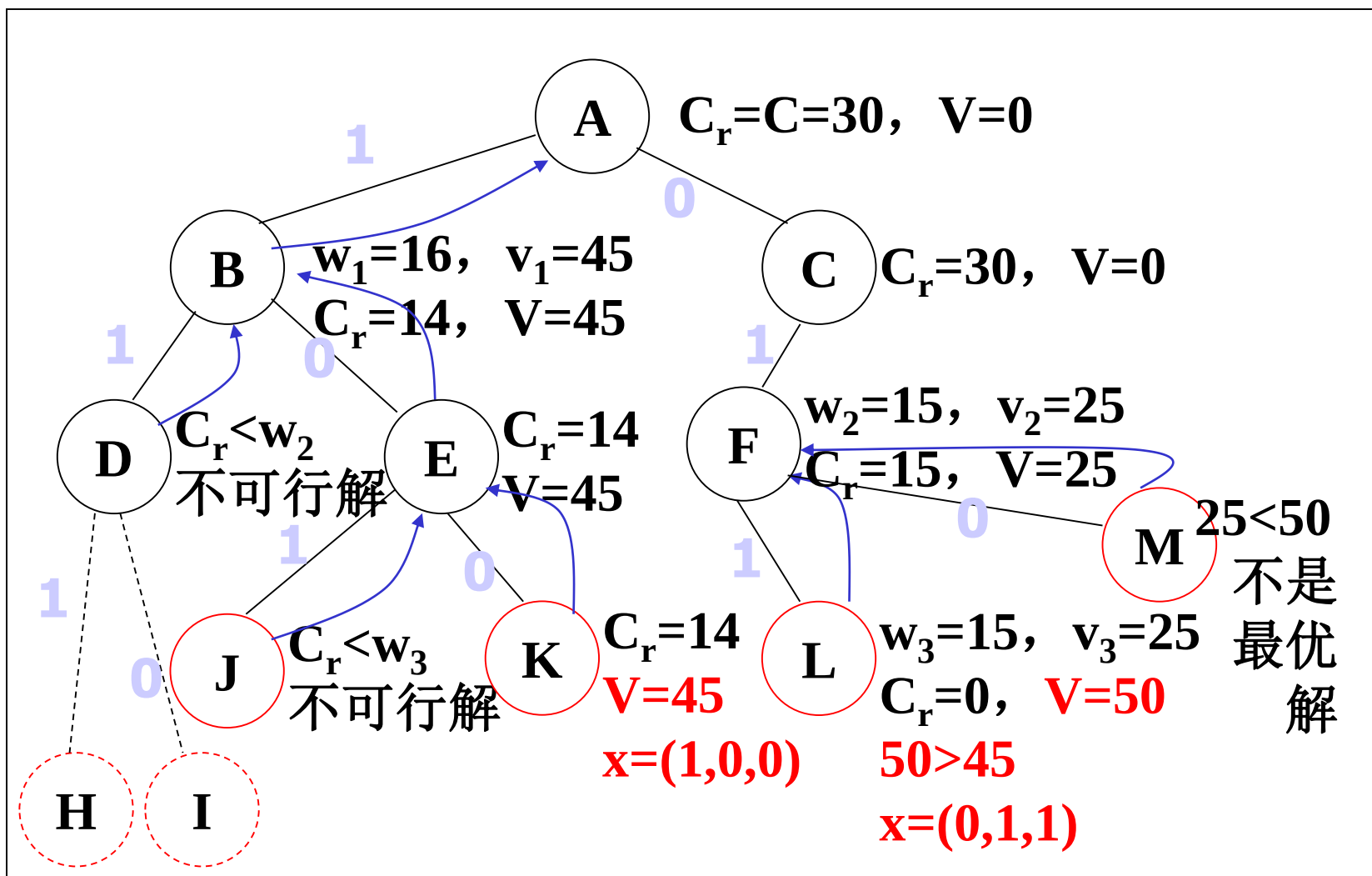
```
void backtrack (int t)  
{  
  if (t>n) output(x);  
  else  
    for (int i=t;i<=n;i++)  
    {  
      swap(x[t], x[i]);  
      if (legal(t))  
        backtrack(t+1);  
      swap(x[t], x[i]);  
    }  
}
```

0-1背包问题

- $n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$
- 开始时, $C_r=C=30$, $V=0$, A为唯一活结点, 也是当前扩展结点
- 扩展A, 先到达B结点
 - $C_r=C_r-w_1=14$, $V=V+v_1=45$
 - 此时A、B为活结点, B成为当前扩展结点
 - 扩展B, 先到达D
 - $C_r < w_2$, D导致一个不可行解, 回溯到B
 - 再扩展B到达E
 - E可行, 此时A、B、E是活结点, E成为新的扩展结点
 - 扩展E, 先到达J
 - $C_r < w_3$, J导致一个不可行解, 回溯到E
 - 再次扩展E到达K
 - 由于K是叶结点, 即得到一个可行解 $x=(1,0,0)$, $V=45$
 - K不可扩展, 成为死结点, 返回到E
 - E没有可扩展结点, 成为死结点, 返回到B
 - B没有可扩展结点, 成为死结点, 返回到A

0-1背包问题

- A再次成为扩展结点，扩展A到达C
 - $C_r=30$, $V=0$, 活结点为A、C, C为当前扩展结点
 - 扩展C, 先到达F
 - $C_r=C_r-w_2=15$, $V=V+v_2=25$, 此时活结点为A、C、F, F成为当前扩展结点
 - 扩展F, 先到达L
 - $C_r=C_r-w_3=0$, $V=V+v_3=50$
 - L是叶结点, 且 $50>45$, 皆得到一个可行解 $x=(0,1,1)$, $V=50$
 - L不可扩展, 成为死结点, 返回到F
 - 再扩展F到达M
 - M是叶结点, 且 $25<50$, 不是最优解
 - M不可扩展, 成为死结点, 返回到F
 - F没有可扩展结点, 成为死结点, 返回到C
 - 再扩展C到达G
 - $C_r=30$, $V=0$, 活结点为A、C、G, G为当前扩展结点
 - 扩展G, 先到达N, N是叶结点, 且 $25<50$, 不是最优解, 又N不可扩展, 返回到G
 - 再扩展G到达O, O是叶结点, 且 $0<50$, 不是最优解, 又O不可扩展, 返回到G
 - G没有可扩展结点, 成为死结点, 返回到C
 - C没有可扩展结点, 成为死结点, 返回到A
- A没有可扩展结点, 成为死结点, 算法结束, 最优解 $X=(0,1,1)$, 最优值 $V=50$



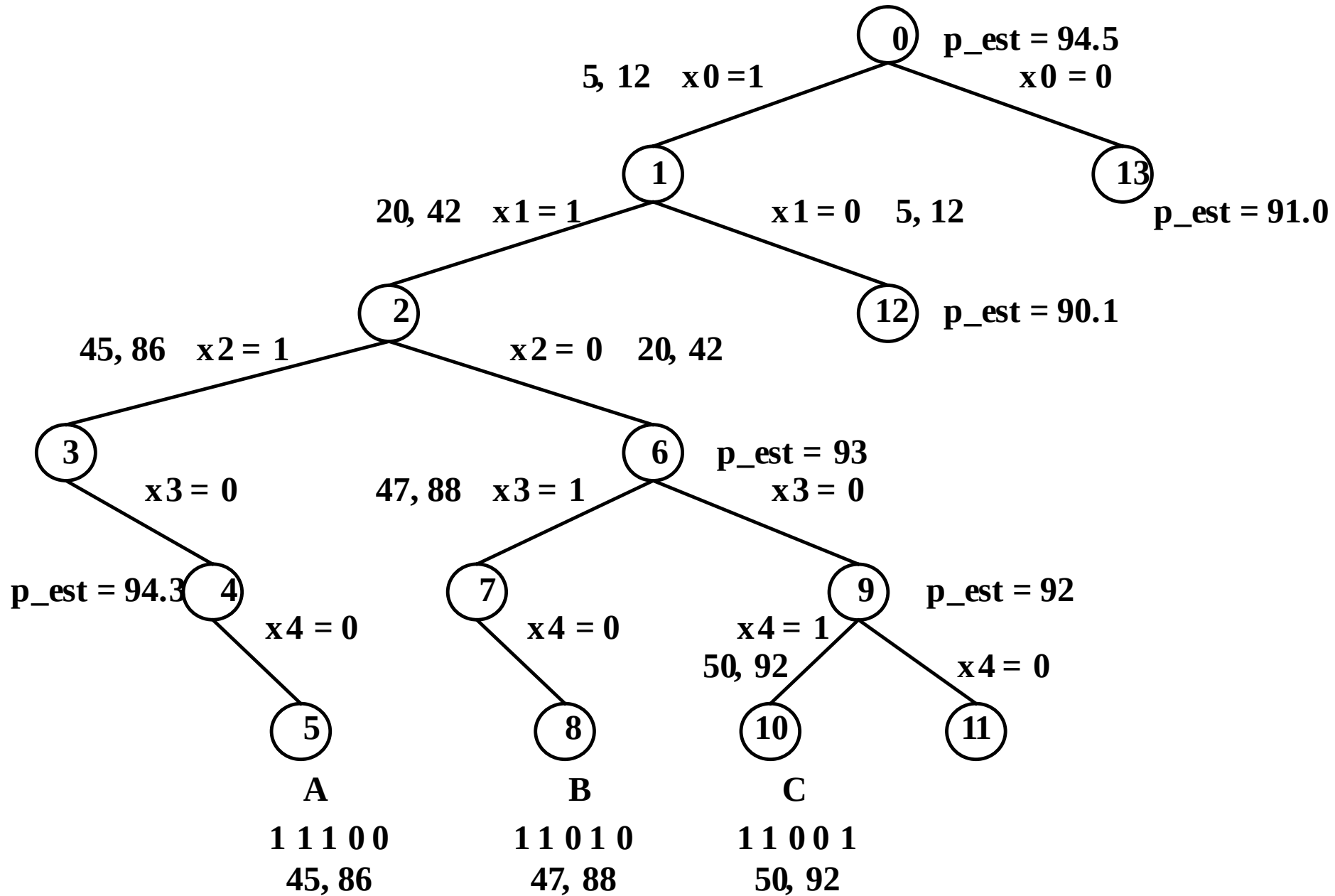
0-1背包回溯 $O(2^n)$

- 输入: n 物品重 $w[1:n]$, 价值 $v[1:n]$, 背包容量 C
- 输出: 装包使得价值最大.
- DP: 物品重量为整数, $O(nC)$ // 输入规模 $\max\{n, \log_2 C\}$

初始: $bestv=cv=cw=0$, $r=\sum_{t=1}^n v[t]$, 执行backtrack(1),
backtrack(t)

1. 若 $t>n$, (若 $cv>bestv$, $bestv=cv$), 返回
2. $r -= v[t]$
3. 若 $cw+w[t] \leq c$, 则 -----// $x[t]=1$
4. | $cw+=w[t]$, $cv+=v[t]$, backtrack($t+1$), $cv-=v[t]$, $cw-=w[t]$
5. 若 $cv + r > bestv$, 则
6. | backtrack($t+1$) -----// $x[t]=0$
7. $r += v[t]$ //可添加提前更新最优值和找最优解, 优于备忘录

C=50, 5, 15, 25, 27, 30, 价值12, 30, 44, 46, 50

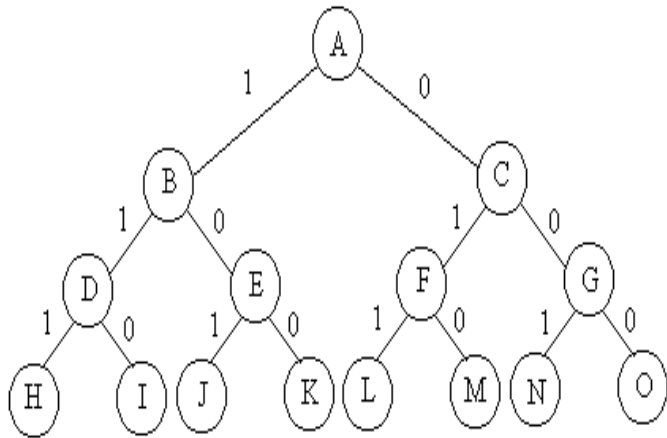


0-1背包问题

0-1背包问题

- 0-1背包问题是一个子集选取问题，在一般情况下，背包问题是NP难的。
- 适合于用子集树表示0-1背包问题的解空间。

0-1背包问题



- 解空间：子集树
- 可行性约束函数：

$$\sum w_i x_i \leq C$$

- 上界函数：
Bound(int i)

0-1背包问题

```
void backtrack (int i)
{
    // 搜索第i层结点
    if (i > n) { // 到达叶结点,更新最优解
        betsp=cp; return;}
    if (cw + w[i] <= c) { // 搜索左子树 x[i] = 1;
        cw += w[i]; //当前重量
        cp+=p[i] //当前价值
        backtrack(i + 1);
        cw -= w[i]; cp-=p[i]; }
    if (bound(i+1)> bestp) //x[i] = 0; //搜索右子
    树
        backtrack(i + 1);
}
```

0-1背包问题

```
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i)
{ // 计算上界
    Typew cleft = c - cw; // 剩余容量
    Typep b = cp;
    // 以物品单位重量价值递减序装入物品
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i];
        b += p[i];
        i++;
    }
    // 装满背包
    if (i <= n) b += p[i]/w[i] * cleft;
    return b;
}
```

b为初始结点到当前结点已经获得的价值

b加上第*i*个物体装满剩余空间获得的价值

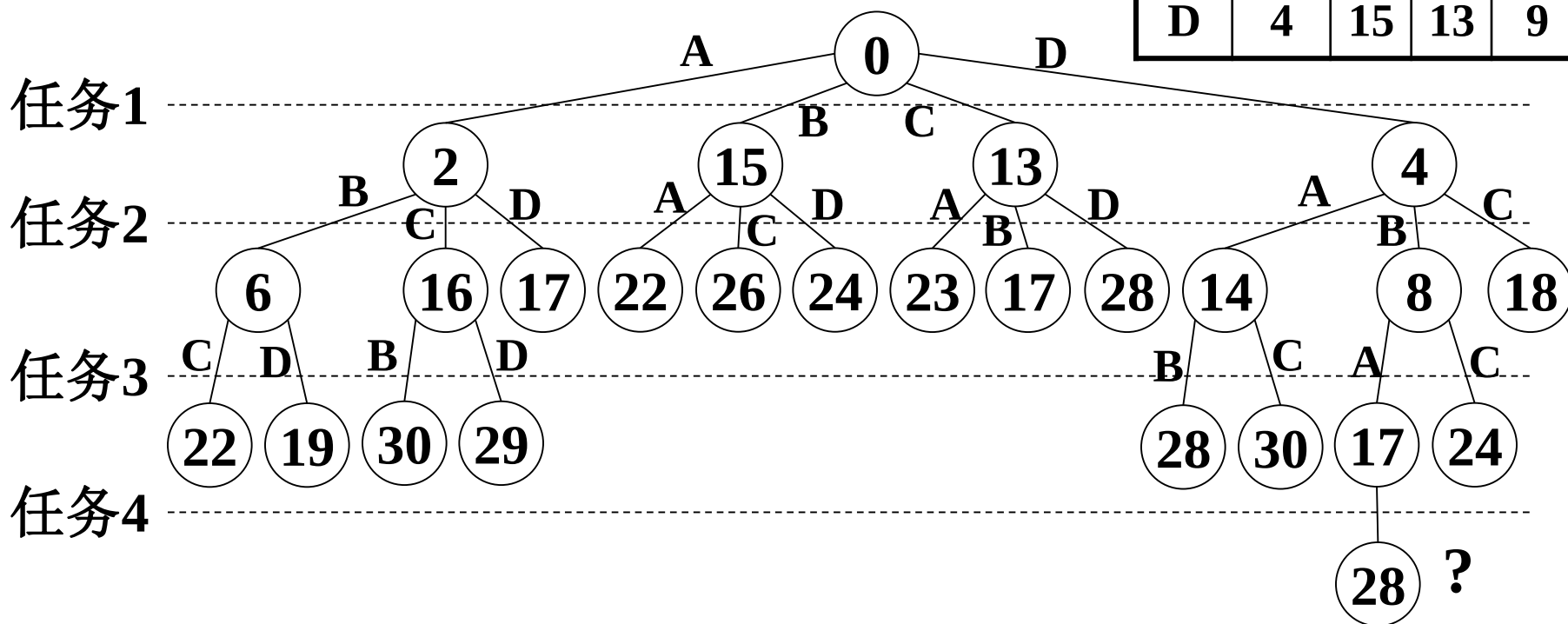
任务分配问题

分支限界-观察：任务分配问题

右表是不同人完成不同任务所需时间
找出总时间最少的分配方案

任务 人	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

任务1:4最少时间: 2, 4, 9, 7



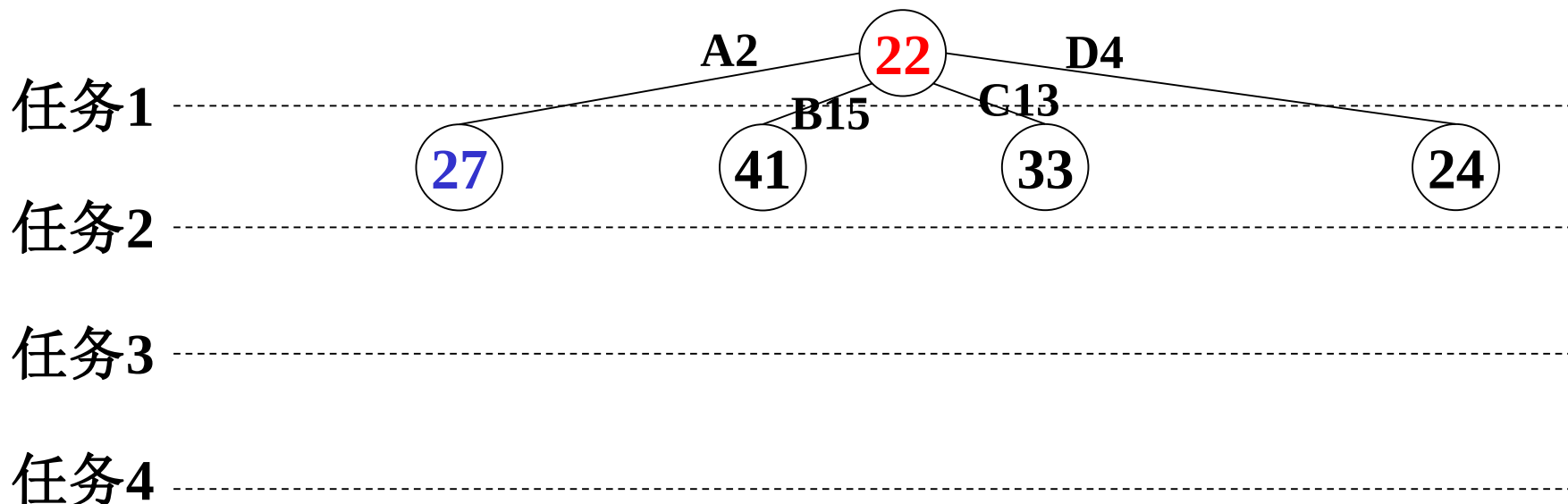
分支限界-任务分配: 时间下界

以(当前时间+剩余**最少时间**)为关键值扩展新的节点

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9



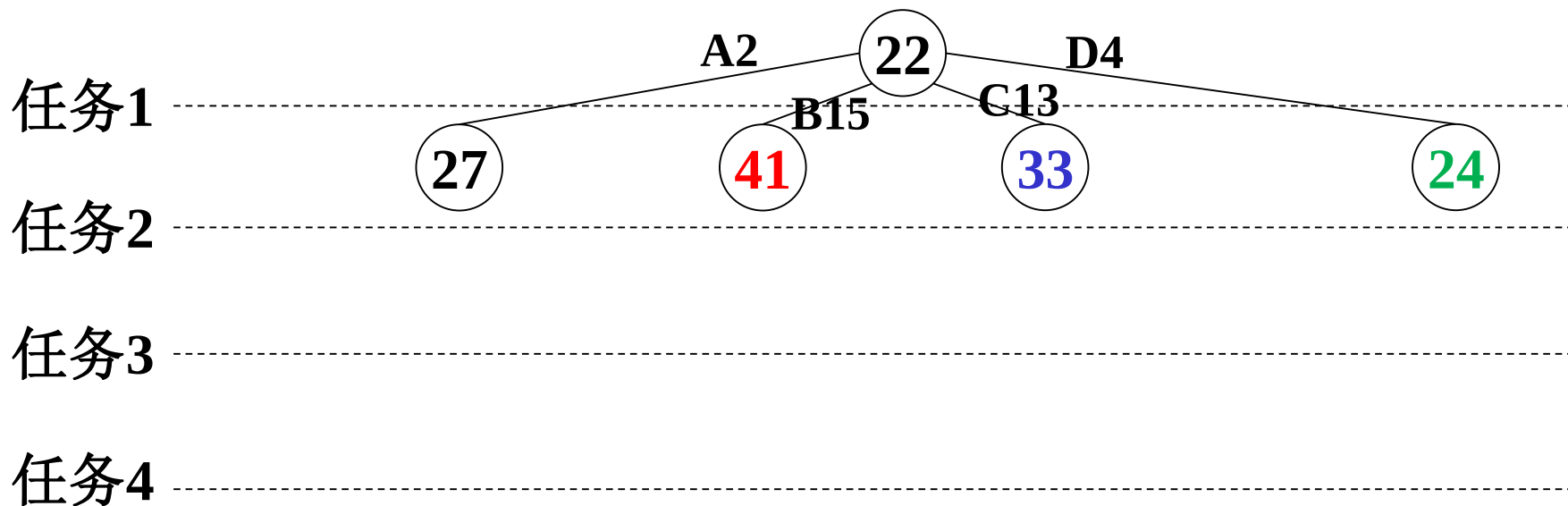
分支限界-任务分配: 时间下界

以(当前时间+剩余**最少时间**)为关键值扩展新的节点

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9



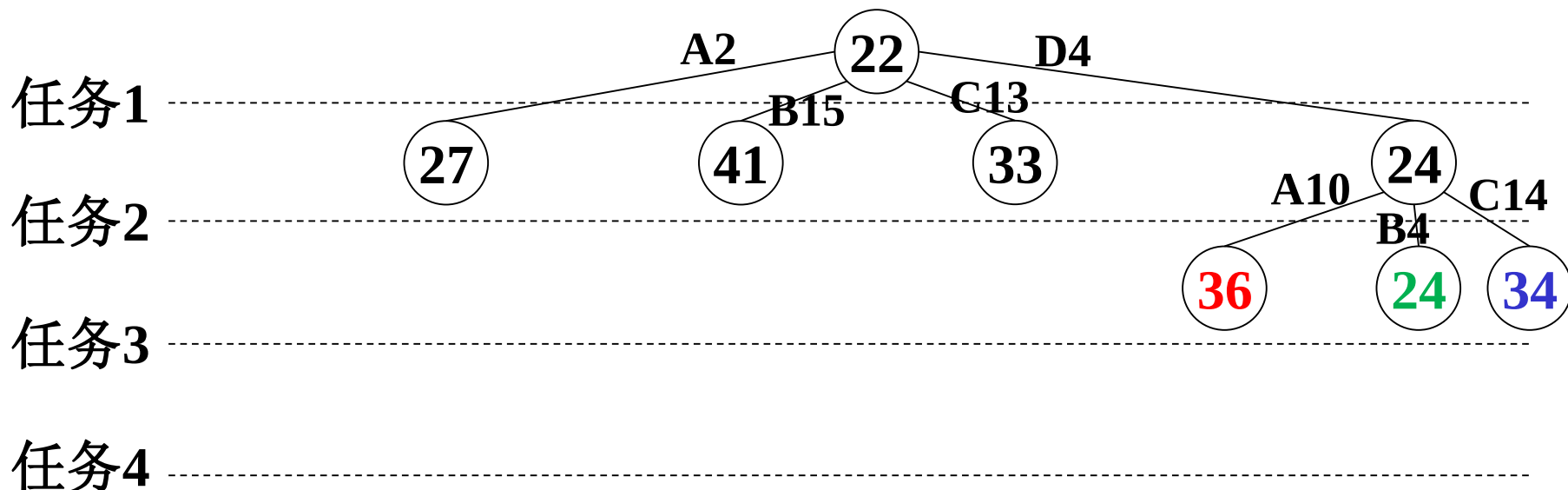
分支限界-任务分配: 时间下界

以(当前时间+剩余**最少时间**)为关键值扩展新的节点

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

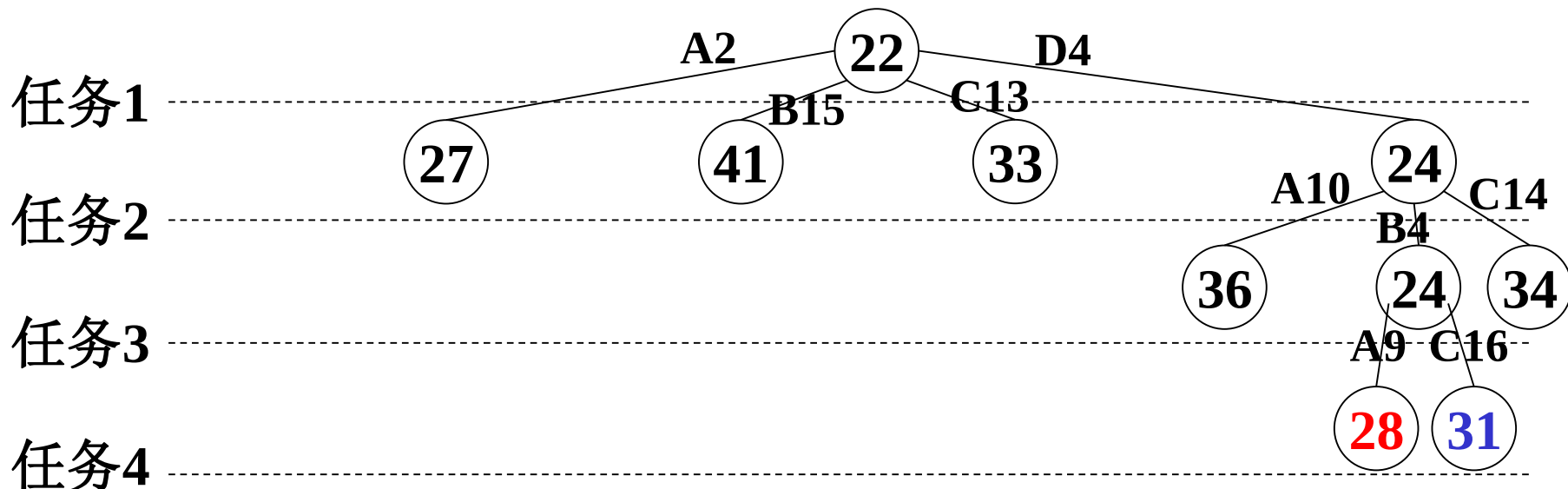


分支限界-任务分配: 时间下界

以(当前时间+剩余**最少时间**)为关键值扩展新的节点

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9



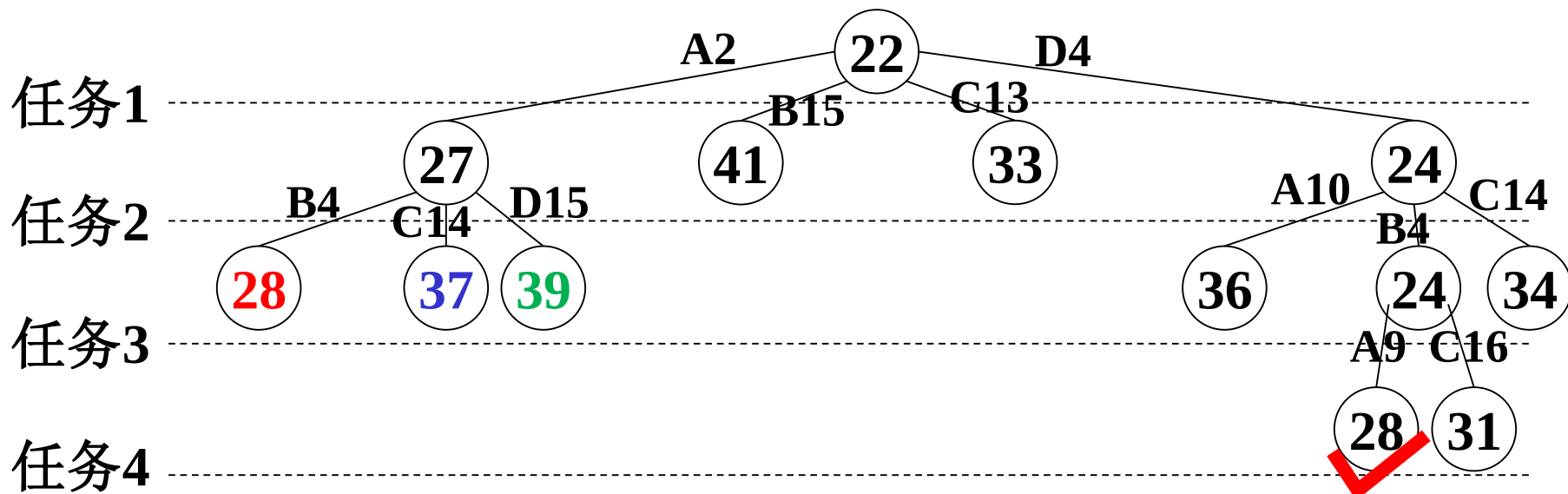
分支限界-任务分配: 时间下界

以(当前时间+剩余**最少时间**)为关键值扩展新的节点

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

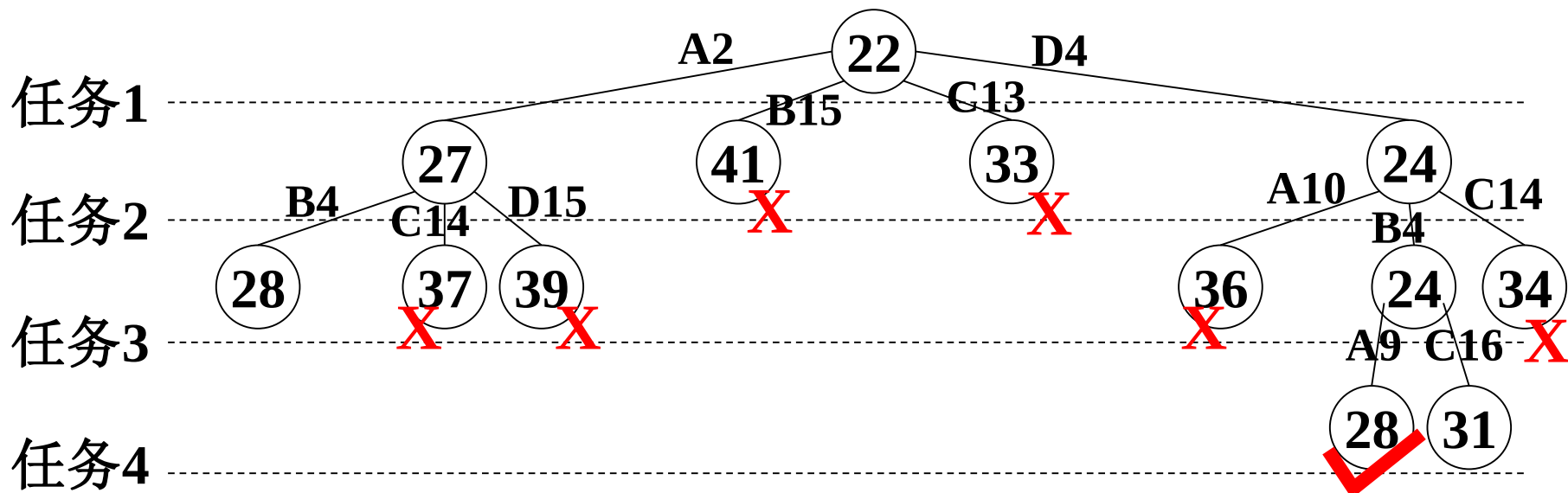
	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9



分支限界-任务分配: 时间下界

	1	2	3	4
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

下界: 当前时间+剩余最少时间



第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n 皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

装载问题

回溯算法设计步骤

1. 定义问题的解空间
2. 确定易于搜索的解空间结构
3. 设计约束和限界函数
4. 以深度优先方式搜索解空间

backtrack(t)

1. 若 $t > n$, 则判断 记录 返回 //记录更新
2. 对 $i = f(n,t) : g(n,t)$ //第 t 层扩展节点的起止编号
3. | $x[t]=h(i)$, //第 t 层的第 i 个可选值
4. | 若 $C(t)$ 且 $B(t)$, 则 **backtrack(t+1)** //剪枝

1. 问题描述

- 有一批共 n 个集装箱要装上2艘载重量分别为 $C1$ 和 $C2$ 的轮船，其中集装箱 i 的重量为 w_i ，且 $\sum w_i \leq C1 + C2$
- 装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这2艘轮船。如果有，找出一种装载方案。

装载问题

- n 件货物(重 $w[1:n]$)装两艘船(载重量 c_1, c_2), $\sum_{i=1}^n w[i] \leq c_1 + c_2$, 是否有装载方案.
- 讨论过类似问题: 0-1背包, 分数背包, 最优装载
- 装载方案: 尽可能装满第1艘, 剩余的装第2艘
- 尽可能装满第1艘等价于下面变形的0-1背包

$$\begin{aligned} & \max \sum_{i=1}^n w[i]x[i] \\ & \text{s.t.} \sum_{i=1}^n w[i]x[i] \leq c \\ & x[i] \in \{0, 1\}, 1 \leq i \leq n \end{aligned}$$

本章以此为装载问题
讨论回溯算法

样例: $w=[16,15,15]$, $c=30$

2. 问题分析

- 容易证明，如果一个给定装载问题有解，则采用下面的策略可得到最优装载方案：
 - (1) 首先将第一艘轮船尽可能装满；
 - (2) 将剩余的集装箱装上第二艘轮船。

2. 问题分析

- 将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近。

$$\max \sum_{i=1}^n w_i x_i$$

■由此可知，**装载问题等价于以下特殊的0-1背包问题。**

$$\text{s.t. } \sum_{i=1}^n w_i x_i \leq c$$

$$x_i \in \{0,1\}, 1 \leq i$$

用回溯法解装载问题的 $O(2^n)$ 计算时间算法。在某些情况下该算法优于动态规划算法。

3. 算法设计

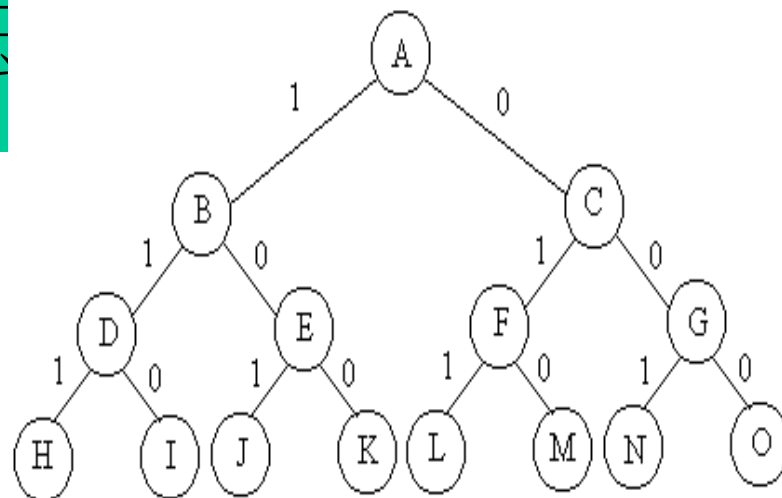
- 解空间：子集树
- 可行性约束函数(选择当前元素):

$$\begin{aligned} \mathbf{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq c_1 \\ & x_i \in \{0, 1\}, 1 \leq i \leq n \end{aligned}$$

- 上界函数
 - 当前载重量 cw +剩余集装箱的重量 $r \leq$ 当前最优载重量 $bestw$

问题：设置上界函数的作用是什么？

3. 算法



- 上界函数:

- 用于剪去不含最优解的子树，从而提高算法在平均情况下的运行效率。
- 设Z是解空间树第i层上的当前扩展结点，**cw**是当前载重量，**R**是剩余集装箱的重量，**bestW**是当前最优载重量，则当
 $CW+R \leq BestW$ 时，剪去Z的右子树。

装载问题 $w[1:n], c$

解空间: $\{x[i] \in \{0,1\}, 1 \leq i \leq n\}$ // 01串

解空间结构: 子集树, 1左0右

bestw, **cw**: 最佳质量, 当前质量

r: 当前剩余质量

$$\max \sum_{i=1}^n w_i x_i$$

$$\sum_{i=1}^n w_i x_i \leq c$$

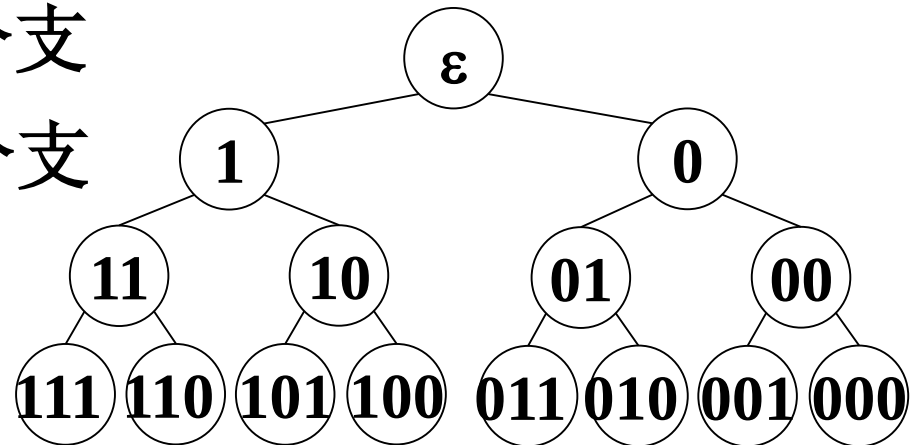
$$x_i \in \{0,1\}, 1 \leq i \leq n$$

$$cw \leq c \quad // \text{约束条件}$$

$$cw + r > \text{bestw} \quad // \text{限界条件}$$

$$cw + w[i] \leq c \quad // \text{用于左分支}$$

$$cw + r > \text{bestw} \quad // \text{用于右分支}$$



子集树回溯模型

backtrack(int t) //搜索到树的第t层

1. 若 $t > n$, 判断 记录 返回
2. 对 $i = 0 : 1$
3. | 若 满足 **Constraint(t)** 和 **Bound(t)**
4. | 则 **backtrack(t+1);**

Constraint(t) 约束条件

Bound(t) 限界条件

有必要时, 注意还原

backtrack(t)

1. 若 $t > n$, 判断 记录 返回
2. **$r -= w[t]$**
3. 若 $cw + w[t] \leq c$, 则
4. | $cw += w[t]$, **backtrack(t+1)**, $cw -= w[t]$,
5. 若 **$cw + r > bestw$** , 则 **backtrack(t+1)**
6. **$r += w[t]$**

排列树回溯模型

backtrack(int t) //搜索树的第t层

1. 若 $t > n$, 判断 记录 返回

2. 对 $i = t : n$

3. | 交换 $x[t]$ 和 $x[i]$

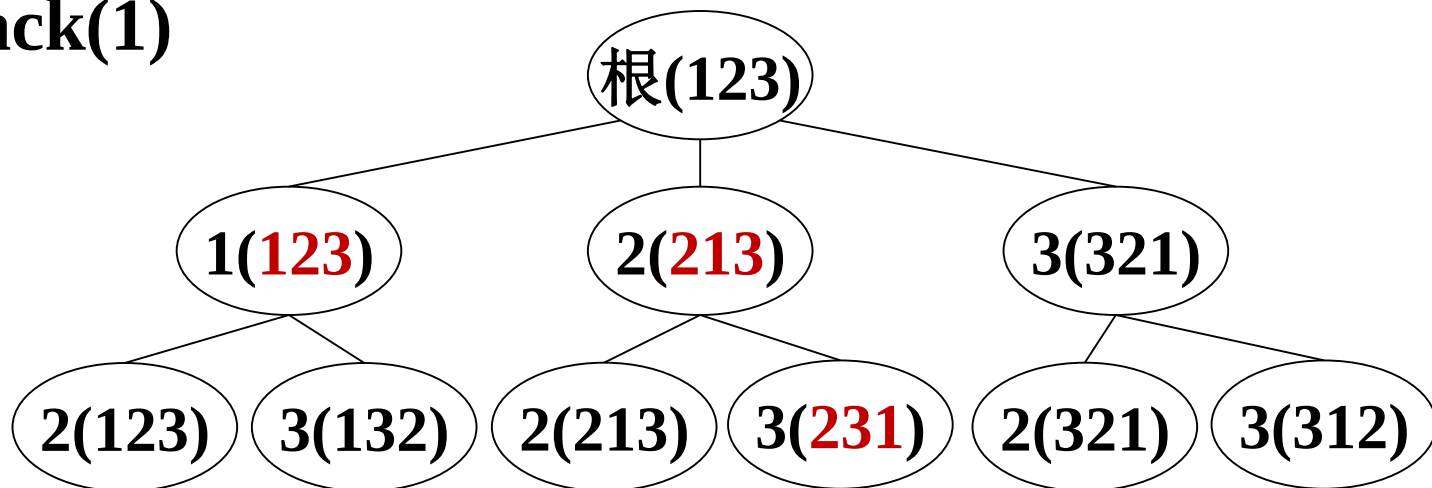
4. | 若 满足 **Constraint(t)** 且 **Bound(t)** //约束和限界条件

5. | 则 **backtrack(t+1)**;

6. | 交换 $x[t]$ 和 $x[i]$

执行**backtrack(1)**

初始 $x[i]=i$



简单回溯：装载问题 $w[1:n], c$

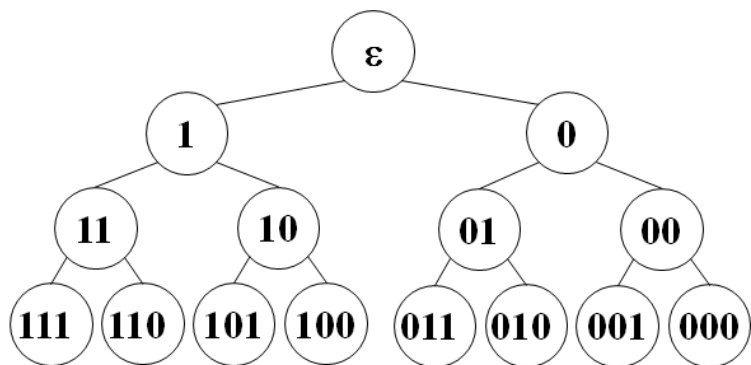
- 在树上进行深度优先搜索, 通常同层结构相同.

backtrack(t)

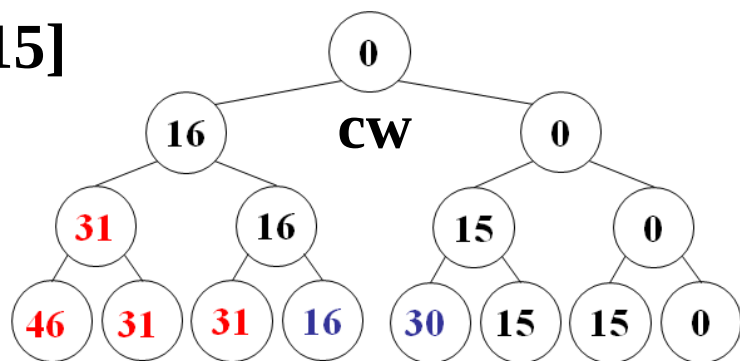
// t:层号, cw:当前重量, bestw:最优重量

- 若 $t > n$, (若 $cw \leq c$ 且 $cw > bestw$, $bestw = cw$.) 返回 ---//记录更新
- $cw += w[t]$, backtrack($t+1$), $cw -= w[t]$, -----//左分支
- backtrack($t+1$)-----//右分支

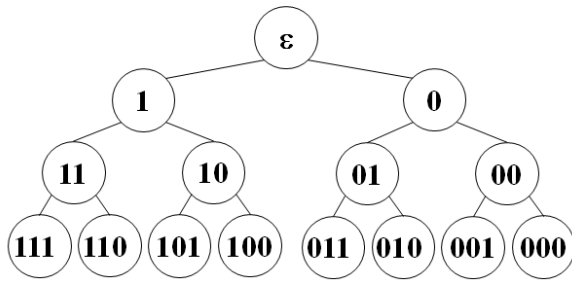
- 初始: $bestw = cw = 0$, 执行 backtrack(1), 简单, 优美



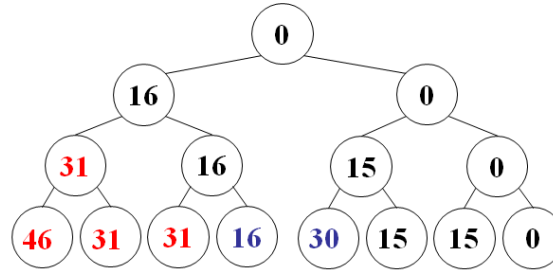
$w=[16,15,15]$
 $c=30$



$w=[16,15,15]$, $c=30$, backtrack(1)



程序隐含的解空间结构



各节点的cw值

$k-b-(t, \text{bestw}, cw)$

//进入第k行前的数据.

//b: 实际对应节点标号

backtrack(t)

1. 若 $t > n$,
2. | 若 $cw \leq c$ 且 $cw > \text{bestw}$,
3. | | $\text{bestw} = cw$
4. | 返回
5. $cw += w[t]$
6. backtrack(t+1) // 进左分支
7. $cw -= w[t]$
8. backtrack(t+1) // 进右分支
9. 返回

1- ϵ -(1,0,0)

1-111-(4,0,46)

5- ϵ -(1,0,0)

2-111-(4,0,46)

6- ϵ -(1,0,16)

4-111-(4,0,46)

1-1-(2,0,16)

7-11-(3,0,46)

5-1-(2,0,16)

8-11-(3,0,31)

6-1-(2,0,31)

1-110-(4,0,31)

1-11-(3,0,31)

2-110-(4,0,31)

5-11-(3,0,31)

4-110-(4,0,31)

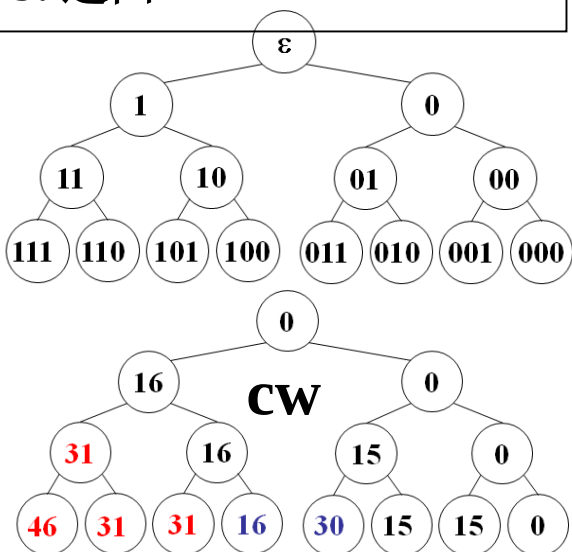
6-11-(3,0,46)

9-11-(3,0,31)

w=[16,15,15], c=30, backtrack(1)

backtrack(t)

1. 若 $t > n$,
2. | 若 $cw \leq c$ 且 $cw > bestw$,
3. | | $bestw = cw$
4. | 返回
5. $cw += w[t]$
6. backtrack(t+1) // 左分支
7. $cw -= w[t]$
8. backtrack(t+1) // 右分支
9. 返回



k-b-(t,bestw,cw) // 进入第k步前的数据. 剪枝?

1-ε-(1,0,0)	9-11-(3,0,31)	7-ε-(1,16,0)	9-01-(3,30,15)
5-ε-(1,0,0)	7-1-(2,0,31)	8-ε-(1,16,0)	7-0-(2,30,15)
6-ε-(1,0,16)	8-1-(2,0,16)	1-0-(2,16,0)	8-0-(2,30,0)
1-1-(2,0,16)	1-10-(3,0,16)	5-0-(2,16,0)	1-00-(3,30,0)
5-1-(2,0,16)	5-10-(3,0,16)	6-0-(2,16,15)	5-00-(3,30,0)
6-1-(2,0,31)	6-10-(3,0,31)	1-01-(3,16,15)	6-00-(3,30,15)
1-11-(3,0,31)	1-101-(4,0,31)	5-01-(3,16,15)	1-001-(4,30,15)
5-11-(3,0,31)	2-101-(4,0,31)	6-01-(3,16,30)	2-001-(4,30,15)
6-11-(3,0,46)	4-101-(4,0,31)	1-011-(4,16,30)	4-001-(4,30,15)
1-111-(4,0,46)	7-10-(3,0,31)	2-011-(4,16,30)	7-00-(3,30,15)
2-111-(4,0,46)	8-10-(3,0,16)	3-011-(4,16,30)	8-00-(3,30,0)
4-111-(4,0,46)	1-100-(4,0,16)	4-011-(4,30,30)	1-000-(4,30,0)
7-11-(3,0,46)	2-100-(4,0,16)	7-01-(3,30,30)	2-000-(4,30,0)
8-11-(3,0,31)	3-100-(4,0,16)	8-01-(3,30,15)	4-000-(4,30,0)
1-110-(4,0,31)	4-100-(4,16,16)	1-010-(4,30,15)	9-00-(3,30,0)
2-110-(4,0,31)	9-10-(3,16,16)	2-010-(4,30,15)	9-0-(2,30,0)
4-110-(4,0,31)	9-1-(2,16,16)	4-010-(4,30,15)	9-ε-(1,30,0)

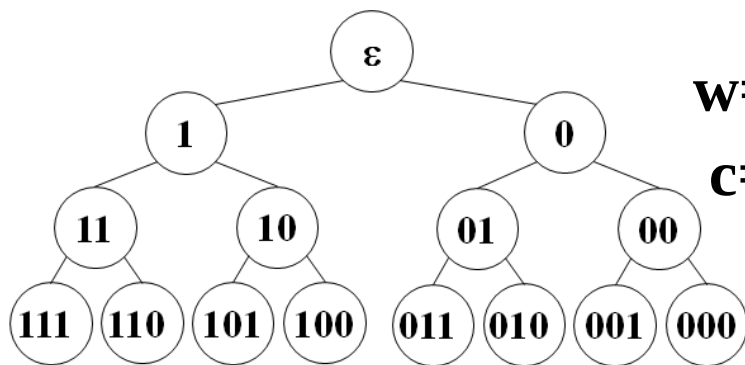
约束条件: 装载问题 $w[1:n], c$

backtrack(t)

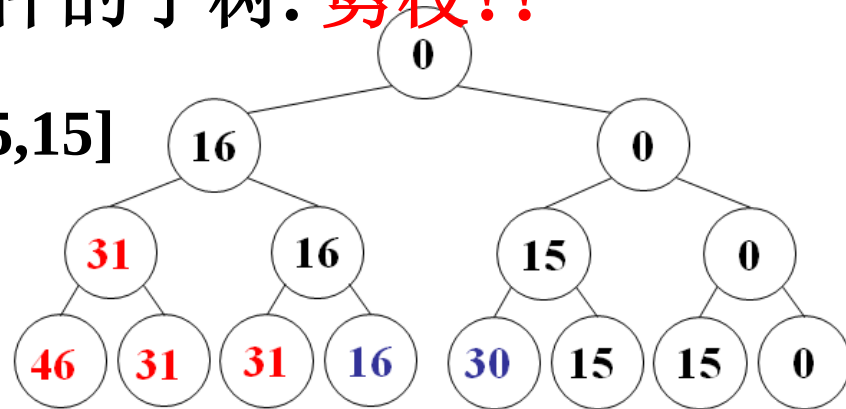
// t:层号, cw:当前重量, bestw:最优重量

1. 若 $t > n$, (若 $cw > bestw, bestw = cw$.) 返回
2. 若 $cw + w[t] \leq c$, 则 -----//约束条件(剪枝)
3. | $cw += w[t]$, backtrack(t+1), $cw -= w[t]$, -----//左分支
4. backtrack(t+1)-----//右分支

约束条件: 剪去不满足约束条件的子树. 剪枝??



$w=[16,15,15]$
 $c=30$



限界条件: 装载问题 $w[1:n], c$

• 初始: $bestw=cw=0$. $r=\sum_{t=1}^n w[t]$ //当前剩余重量

$backtrack(t)$ //t层号, cw , $bestw$

1. 若 $t > n$, (若 $cw > bestw$, $bestw=cw$, $bestx=x$.) 返回

2. $r -= w[t]$

3. 若 $cw + w[t] \leq c$, 则-----//约束条件(剪枝)

4. | $cw += w[t]$, $x[t]=1$, $backtrack(t+1)$, $cw -= w[t]$, //左分支

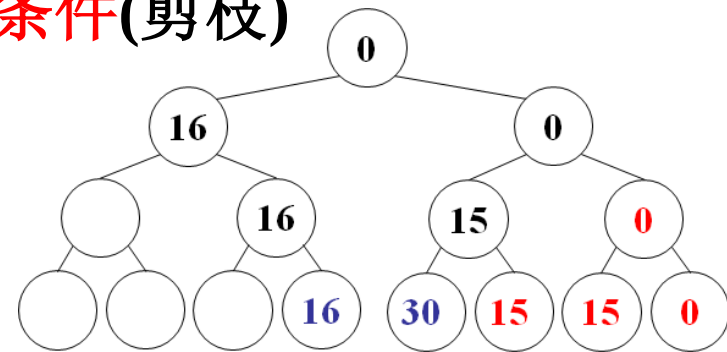
5. 若 $cw + r > bestw$, 则-----//限界条件(剪枝)

6. | $x[t]=0$, $backtrack(t+1)$ //右分支

7. $r += w[t]$ //还原

限界条件: 剪去得不到最优解子树

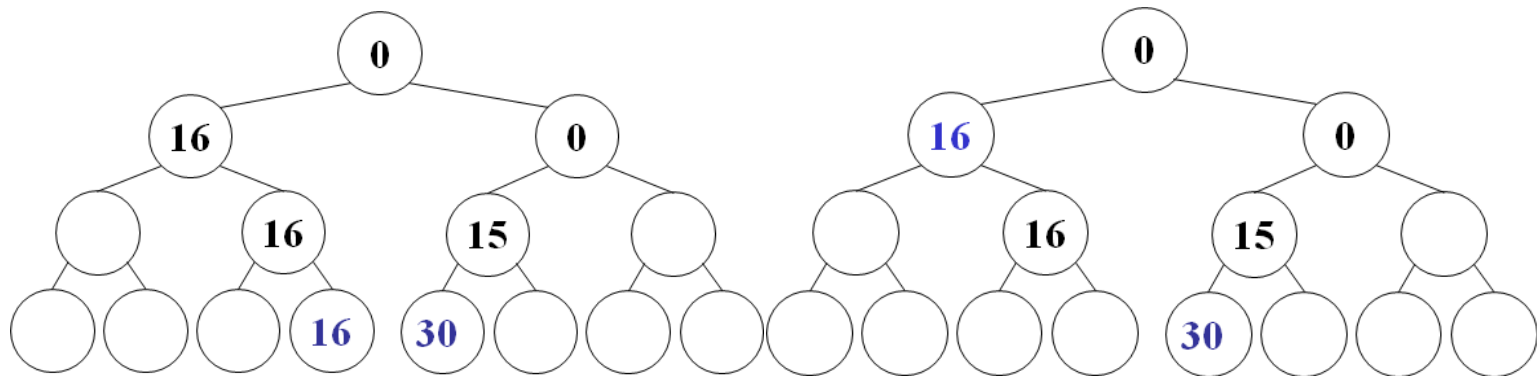
约束条件与限界条件统称为剪枝函数. 执行哪些节点?



提前更新最优值

backtrack(t)

1. 若 $t > n$, 则返回
2. $r += w[t]$
3. 若 $cw + w[t] \leq c$, 则
4. | 若 $cw + w[t] > bestw$, 则 $bestw = cw + w[t]$
5. | $cw += w[t]$, backtrack($t+1$), $cw -= w[t]$ -----// $x[t] = 1$
6. 若 $cw + r > bestw$, 则
7. | backtrack($t+1$) -----// $x[t] = 0$
8. $r += w[t]$



递归回溯(backtracking)

backtrack(t)

1. 若 $t > n$, 则判断 记录 返回 //记录更新
2. 对 $i = f(n,t) : g(n,t)$ //第 t 层扩展节点的起止编号
3. | $x[t]=h(i)$, //第 t 层的第 i 个可选值
4. | 若 $C(t)$ 且 $B(t)$, 则backtrack($t+1$) //剪枝

- $C(t)$: 约束函数 $B(t)$: 限界函数
- 执行backtrack(1)完成搜索
- 判断记录也可能提前.
- 可以迭代回溯.

backtrack(t)

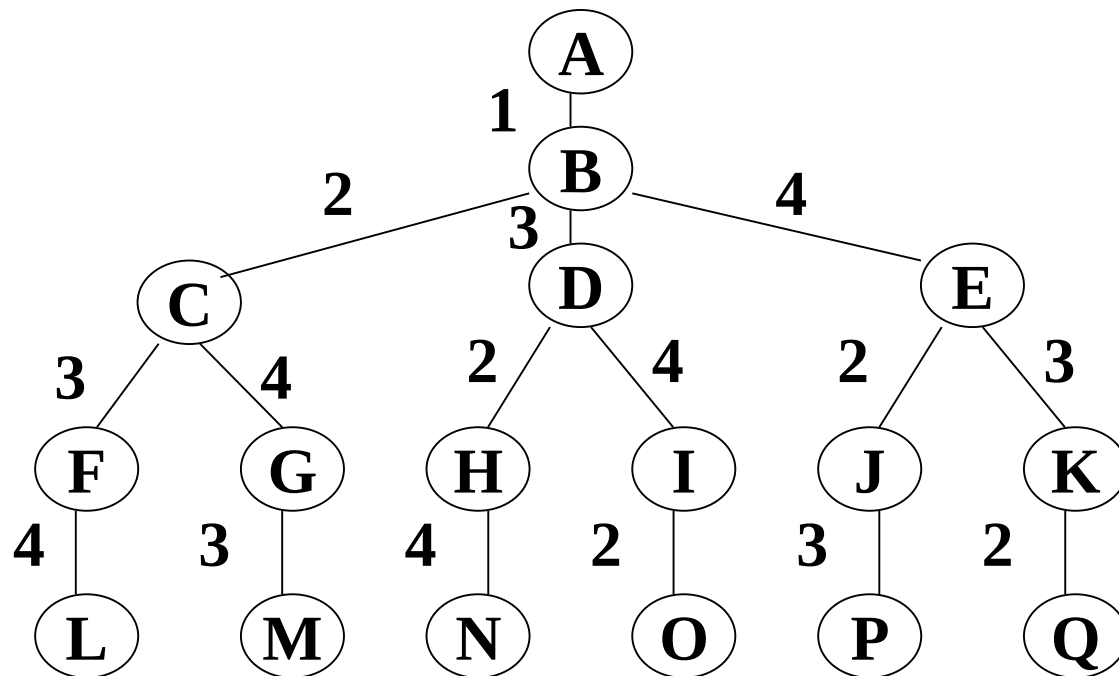
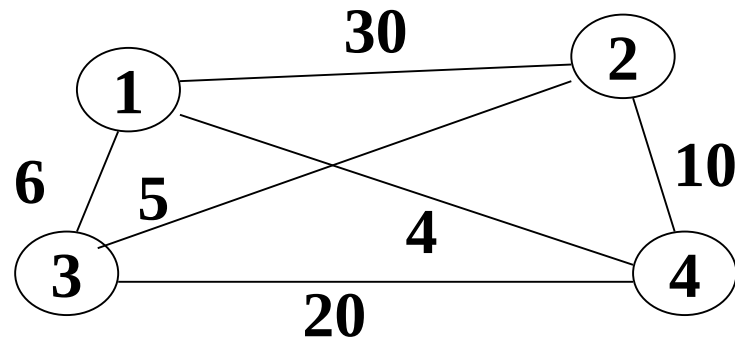
1. 若 $t > n$, 则判断 记录 返回
2. 对 $i = 1 : 0$
3. | $x[t]=i$, $cw+=x[t]*w[t]$
4. | 若 $C(t)$ 且 $B(t)$, 则backtrack($t+1$)
5. | $cw-=x[t]*w[t]$

第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n 皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

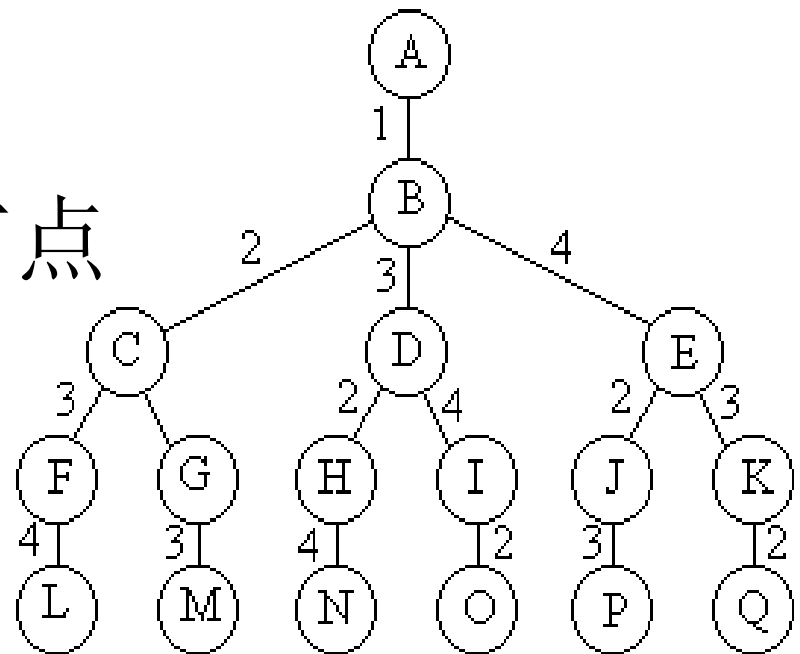
旅行售货员问题

- 问题描述：某售货员要到若干城市去推销商品，一直各城市之间的路程，他要选定一条从驻地出发，经过每个城市一遍，最后回到住地的路线，使总的路程最短。
- 该问题是一个NP完全问题，有 $(n-1)!$ 条可选路线



问题分析

- 旅行商问题
- 含有 $(n-1)!$ 个叶节点



旅行售货员问题(TSP)

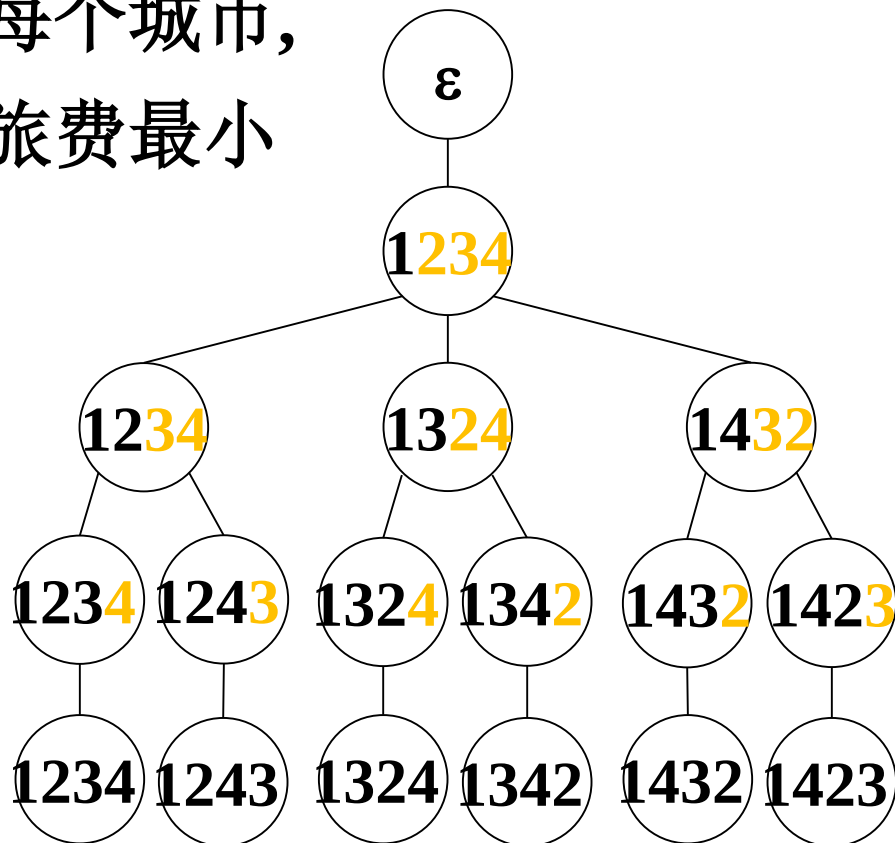
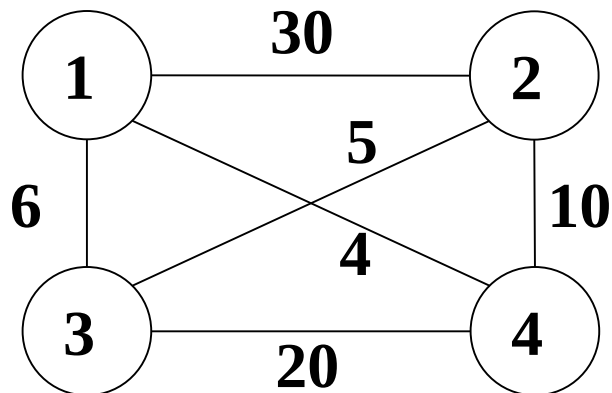
某售货员要到若干城市推销商品

已知各城市间的旅费

选择一条从驻地出发, 经过每个城市,
最后回到驻地的路线, 使总旅费最小

解空间: 全体排列

解空间结构: 排列树



TSP

backtrack(t)

1. 若 $t > n$, 则判断记录 **bestc**, **bestx**, 返回

2. 对 $j = t : n$

3. | 交换 $x[t], x[j]$,

4. | 若 $x[1:t]$ 费用 $< \text{bestc}$, 则

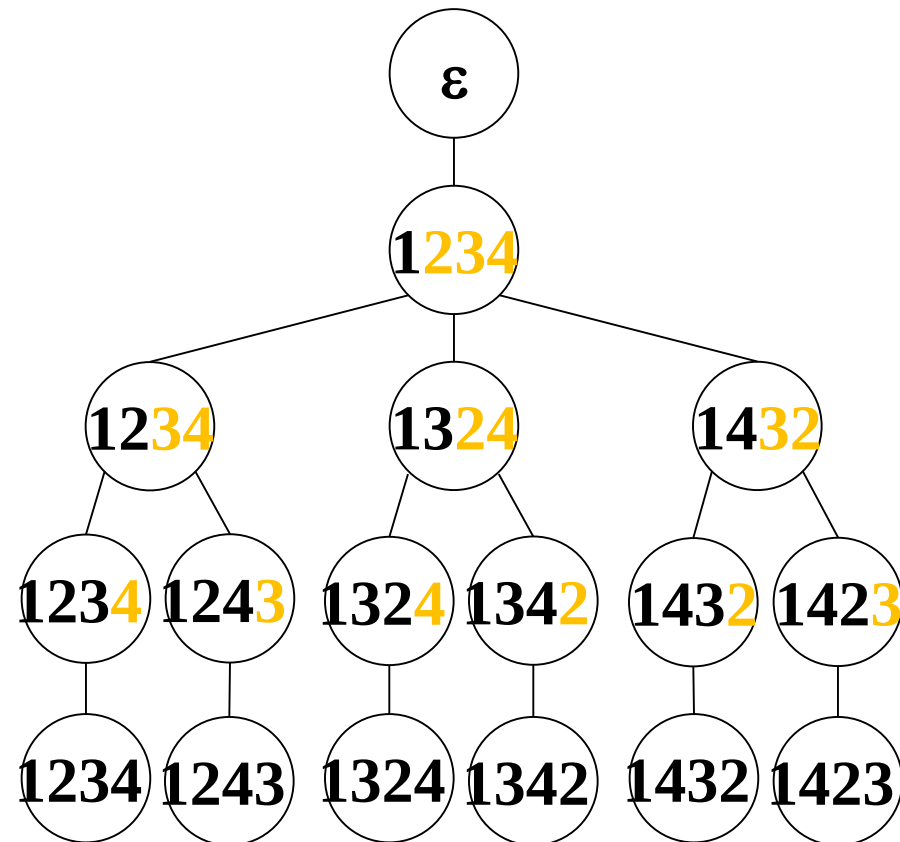
5. | | **backtrace(i+1)**

6. | 交换 $x[i], x[j]$

• 初始 $x[i]=i$, **bestc**=INF,

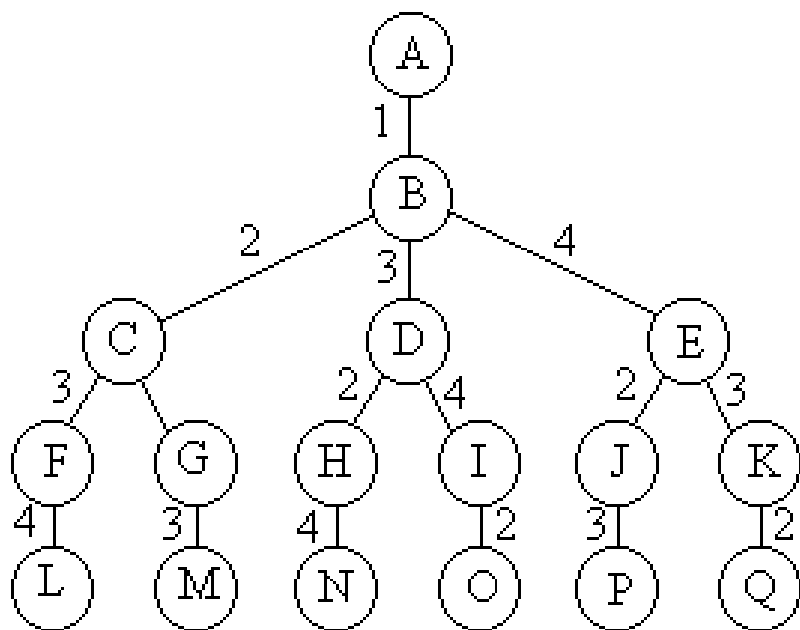
• **backtrack(2)**

• 分支数 $O((n-1)!)$, 时间 $O(n!)$



旅行售货员问题

- 解空间：
- 排列树



遍历排列树需要
 $O(n!)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=t;i<=n;i++)
        {
            swap(x[t], x[i]);
            if (legal(t))
                backtrack(t+1);
            swap(x[t], x[i]);
        }
}
```

旅行售货员问题

- 解空间：排列树

```
template<class Type>
void Traveling<Type>::Backtrack(int i)
{
    if (i == n) {
        if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
(cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc == NoEdge))
            { for (int j = 1; j <= n; j++) bestx[j] = x[j];
              bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];}
    }
    else {
```

旅行售货员问题

```
else {  
    for (int j = i; j <= n; j++)  
        // 是否可进入x[j]子树?  
        if (a[x[i-1]][x[j]] != NoEdge &&  
            (cc + a[x[i-1]][x[i]] < bestc || bestc == NoEdge)) {  
            // 搜索子树  
            Swap(x[i], x[j]);  
            cc += a[x[i-1]][x[i]];  
            Backtrack(i+1);  
            cc -= a[x[i-1]][x[i]];  
            Swap(x[i], x[j]);  
        }  
}
```

旅行售货员问题

复杂度分析

算法backtrack在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次，每次更新bestx需计算时间 $O(n)$ ，从而整个算法的计算时间复杂性为 $O(n!)$ 。